```
* PASS no route out (1.1.1.1:443, 8.8.8.8:53, 9.9.9.9:443)
```

ari

Local agents that keep an offline server healthy, and prove every fix.

Team AirGap AI · Open Agent Hackathon 2026 · Track 03, reasoning architecture

THE PROBLEM

Servers on isolated networks break at 3am, and nobody is watching.

For the on-call admin of a utility’s air-gapped operations servers, or any lab, plant or ship network with no internet.

No phone home

High assurance networks block the cloud agents and monitoring most teams rely on.

Found hours late

A stopped service or a bad config waits for the next person to log in.

AI is not trusted

A model that can run any command is the last thing these teams will allow near production.

THE SOLUTION

Fix only what is approved. Prove it. Undo it. Or hand it to a human.

ari repairs from a short catalogue of reversible fixes. Every fix has a done test and an undo. Everything else is escalated with the evidence.

AGENT WORKFLOW

Six roles, one supervised loop

Explorer

Checks process and health each cycle, writes rows, flags drift from baseline

Investigator

Queries Zetaris for the rows, writes a cause that cites them

Fixer

Picks one catalogue entry, by rule or by model, with a one line reason

Scout

Guards the pick and signs a contract for exactly one repair

Verifier

Checks the contract, applies, runs the done test, undoes on failure

Reporter

One screen: fixed, left alone, and why

A failed done test goes back to the investigator once with the failure as new evidence; a second failure escalates.

A REAL RUN

The service is killed. ari restarts it and proves it.

```
14:11:12 [ ARI ] finding f1: process_missing (rows 11,12)

14:11:21 [ ARI : nightshift ] f1 picked "restart-service": finding is process_missing and
restart-service handles process_missing

14:11:21 [ ARI ] scout contract c-20261023T141121-001 for f1: intent toyd.process_missing,
confidence 0.90, risk low

14:11:21 [ ARI ] result restart-service ok

14:11:21 [ ARI ] resolved f1 with restart-service
```

Teal: the model's words · Orange: Scout's contract

Model words only ever appear on tagged lines. The investigator cited rows it read back from Zetaris.

WHEN THE MODEL IS WRONG

The model picked the wrong repair. Scout refused it, and nothing ran.

```
14:14:06 [ ARI : nightshift ] f1 picked "restore-config": health_slow indicates a health issue, and
restore-config handles health_failing

14:14:06 [ ARI ] scout contract c-20261023T141406-001 for f1: intent toyd.health_slow, confidence
0.00, risk high (scope.mismatch)

14:14:06 [ ARI ] escalate f1: scout refused: scope.mismatch (rows 11,12)
```

Teal: the model's words · Orange: Scout's contract

The model never gets a veto. A memory leak has no approved repair, so it goes to a human.

WHAT THE HUMAN SEES

Status at a glance, a report for the next shift

```
$ ari status
ari: running (pid 58887, run 20261023T131036Z)
last cycle: 11 at 2026-10-23T13:11:35Z (3s ago)
last fix: f1 restart-service resolved at 2026-10-23T13:11:20Z
left alone: 0 finding(s); see ari report
```

```
$ ari report
fixed:
f1 process_missing via restart-service (1 attempt(s), cycle 6, …)
before: row 11 process alive=false … row 12 health status=0
after: row 14 health latency_ms=0 status=200
scout: c-20261023T130715-001 risk low for restart-service
```

VISIBLE BY DESIGN

A real air gap, and nothing hidden

Fails closed

```
* PASS data folder writable
* PASS no route out (1.1.1.1:443, 8.8.8.8:53, 9.9.9.9:443)
* PASS model endpoint answers
* PASS zetaris answers SELECT 1
```

systemd enforces it: IPAddressDeny=any, IPAddressAllow=localhost.

Rules only or a local model: nothing leaves at all. With Nightshift, only the model call does, through one tunnel.

Says what it does

- Login banner: it is here, what it may do, how to stop it
- Every fix announced before it runs, then its result
- ari watch, ari status, ari report
- ari stop halts it before its next action, and it stays stopped

ZETARIS

The investigator does not trust its memory. It asks Zetaris.

Write

Every observation becomes a row in a postgres evidence db, registered in Zetaris as ari_evidence with a read only role.

Read

For a finding, Lightning SQL returns its rows plus the health rows of the cycles before it, as the baseline.

Fail closed

Zetaris down, or a cited row missing: ari escalates and runs nothing.

```
SELECT id, cycle, kind, data FROM ari_evidence.evidence_rows WHERE run = … AND (id IN (…) OR (kind = 'health' AND cycle ⩾ c - 3 AND cycle < c))
```

METERLESS

Nothing runs without a valid Scout contract

Scout Intent, built in Go from the spec

- Intent registry from the catalogue
- Guard: unknown entry, wrong symptom, injection in the reason, budget spent
- Execution contract: HMAC signed, 5 minute TTL, exactly one capability
- Verifier refuses a bad signature, an expired or high risk contract, or a repair outside its scope
- A retry gets a re planned contract with the first as its parent

Nightshift

The fixer’s model provider. Headless Nightshift runtime, routing each pick to NVIDIA’s cloud models; the log records which model answered and its receipt. ari reaches it through a restricted tunnel on its own localhost, so ari itself never leaves the machine. All three scenarios pass with it.

TECHNICAL STACK

One binary, nothing to install offline

Part	What it is
ari	Go, one static binary: run, status, watch, stop, report
Evidence	Postgres on Zetaris’s docker network, read through Zetaris
Zetaris	Self hosted platform from the hackathon bundle
Scout Intent	Meterless engine, Go, in the verifier’s path
Model	Nightshift routing to NVIDIA cloud models; rule fixer when none is set
Air gap	systemd unit: localhost only, read only system, one writable folder
Target	toyd, a small service with planted faults

WHAT IS UNIQUE

The model never gets a veto.

Evidence you can query

Every cause cites rows that Zetaris returned, not text the model wrote.

A contract to act

Scout’s signed contract is the only thing the verifier accepts.

Every fix reversible

A catalogue entry without an undo and a done test is rejected on load.

Fenced in for real

ari proves it has no route out before it starts; only the model call leaves, through Nightshift.

WHY NOT ZABBIX OR ANSIBLE

They can restart a service too. ari adds proof and refusal.

Zabbix, Ansible

Run a command or playbook when a trigger fires, such as restarting an application that stops answering. Proven tools, and many isolated sites already use them.

ari adds

- A signed Scout contract for every action, one capability, five minute expiry
- A cause that cites evidence rows read back from Zetaris
- A done test, an undo, one retry, then a human with the evidence
- Refusal: nothing outside the catalogue runs, even when the model asks

NEXT

Where ari goes from here

More services

Catalogues for real daemons: disk, certificates, queues.

Memory across runs

H-MEM so a repeat fault starts from what worked last time.

Signed catalogues

Only repairs a team has approved and signed can load.

Model words the report

A plain summary for the next human, still tagged.

Code, setup and recorded runs: repository link in the submission